

Week-6 (DAY-30) Hands-On – Keerthivasan R

Problem 1 (Level-1): Student Registration & Display

Scenario:

A small institute wants a simple web page where users can enter student details and see the submitted data on another page.

Requirements:

1. Create a form to accept:
 - **Student Name**
 - **Age**
 - **Course**
2. Submit the form using HttpPost
3. Redirect to another action method to display entered data
4. Pass data using ViewBag OR ViewData

Technical Constraints

1. Use Attribute-based routing
2. Do NOT use Model or Database
3. Use only ViewBag/ViewData for state management
4. Use separate actions for:
 - GET → Display form
 - POST → Handle submission

Expectations

1. Clean separation of GET and POST actions
2. Correct usage of ViewBag/ViewData
3. Proper routing using attributes
4. Data displayed correctly after submission

Learning Outcome

1. Understanding of HttpGet vs HttpPost
2. Basics of state management using ViewBag/ViewData
3. Hands-on with attribute routing

CODE:

Controller/ `StudentController.cs`

```
using Microsoft.AspNetCore.Mvc;

namespace StudentRegistration.Controllers
{
    public class StudentController : Controller
    {
        // GET: Student/Register
        [HttpGet]
        public IActionResult Register()
        {
            return View();
        }

        // POST: Student/Register
        [HttpPost]
        public IActionResult Register(string studentName, int age, string course)
        {
            ViewBag.StudentName = studentName;
            ViewBag.Age = age;
            ViewBag.Course = course;

            return View("Display");
        }

        // GET: Student/Display
        public IActionResult Display()
        {
            return View();
        }
    }
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios,
    see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Student}/{action=Register}/{id?}");

app.Run();
```

View/ Student/ Display.cshtml

```
@{
    Layout = null;
}

<!DOCTYPE html>
<html>
<head>
    <title>Student Details</title>
    <style>
        body {
            font-family: Arial;
            margin: 50px;
        }

        .container {
            width: 400px;
            padding: 20px;
            border: 1px solid #ccc;
            border-radius: 5px;
            background-color: #f9f9f9;
        }

        .success {
            color: green;
            font-weight: bold;
        }

        .details {
            margin: 20px 0;
            padding: 15px;
            background-color: white;
            border-left: 4px solid green;
        }

        .back-link {
            display: inline-block;
            margin-top: 10px;
            color: #007bff;
            text-decoration: none;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>☑ Registration Successful</h2>

        @if (ViewBag.StudentName != null)
        {
            <div class="details">
                <p><strong>Student Name:</strong> @ViewBag.StudentName</p>
                <p><strong>Age:</strong> @ViewBag.Age</p>
                <p><strong>Course:</strong> @ViewBag.Course</p>
            </div>
            <p class="success">✓ Student has been registered successfully!</p>
        }
        else
        {
            <p>No student data found.</p>
        }

        <br />
        <a href="/Student/Register" class="back-link">← Register Another Student</a>
    </div>
</body>
</html>
```

View/ Student/ Register.cshtml

```
@{
    Layout = null;
}

<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
    <style>
        body {
            font-family: Arial;
            margin: 50px;
        }
        .container {
            width: 400px;
            padding: 20px;
            border: 1px solid #ccc;
            border-radius: 5px;
        }
        .form-group {
            margin-bottom: 15px;
        }
        label {
            display: inline-block;
            width: 100px;
        }
        input {
            padding: 5px;
            width: 250px;
        }

        button {
            padding: 8px 20px;
            background-color: #007bff;
            color: white;
            border: none;
            cursor: pointer;
            border-radius: 3px;
        }

        button:hover {
            background-color: #0056b3;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>📝 Student Registration Form</h2>

        <form method="post">
            <div class="form-group">
                <label>Student Name:</label>
                <input type="text" name="studentName" required />
            </div>
            <div class="form-group">
                <label>Age:</label>
                <input type="number" name="age" required min="1" max="120" />
            </div>
            <div class="form-group">
                <label>Course:</label>
                <input type="text" name="course" required />
            </div>
            <button type="submit">Submit Registration</button>
        </form>
    </div>
</body>
</html>
```

OUTPUT:

to exit full screen, press and hold Esc

Student Registration Form

Student Name:

Age:

Course:

[Submit Registration](#)

Registration Successful!

Student Name: Ranjith

Age: 16

Course: Computer Science

✓ Student has been registered successfully!

[← Register Another Student](#)

<https://localhost:7022/Student/Registered>

Problem 1 (Level-1): Simple Calculator

Scenario:

Build a simple calculator web page that performs addition of two numbers.

Requirements:

1. Accept two numbers using a form
2. Submit using **HttpPost**
3. Display result on the same or another page

3. Pass result using **ViewData**

Technical Constraints

1. Use **Attribute routing**
2. No JavaScript (pure server-side processing)
3. No Model binding (use form collection or parameters)

Expectations

1. Correct calculation logic
2. Proper HttpPost handling
3. Result displayed using ViewData

Learning Outcome

1. Handling user input via forms
2. Passing computed values using ViewData
3. Understanding request lifecycle

CODE:

```
Controller/ CalculatorController.cs

using Microsoft.AspNetCore.Mvc;

namespace SimpleCalculator.Controllers
{
    public class CalculatorController : Controller
    {
        // GET: Calculator/Index
        [HttpGet]
        public IActionResult Index()
        {
            return View();
        }

        // POST: Calculator/Add
        [HttpPost]
        public IActionResult Add(double num1, double num2)
        {
            double result = num1 + num2;

            ViewData["Number1"] = num1;
            ViewData["Number2"] = num2;
            ViewData["Result"] = result;

            return View("Result");
        }
    }
}

Program.cs

var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios,
    see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Calculator}/{action=Index}/{id?}");

app.Run();
```

Views/Result.cshtml

```
@{
    Layout = null;
}

<!DOCTYPE html>
<html>
<head>
    <title>Calculation Result</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 50px;
            background-color: #f0f0f0;
        }

        .container {
            width: 450px;
            padding: 30px;
            background-color: white;
            border-radius: 10px;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
            margin: auto;
            text-align: center;
        }

        h2 {
            color: #333;
        }

        .result-box {
            background-color: #e8f5e9;
            padding: 30px;
            border-radius: 8px;
            margin: 20px 0;
            border-left: 5px solid #4caf50;
        }

        .calculation {
            font-size: 28px;
            font-weight: bold;
```

```

        color: #2e7d32;
        margin: 15px 0;
    }

    .formula {
        font-size: 18px;
        color: #555;
        margin: 10px 0;
    }

    .back-link {
        display: inline-block;
        margin-top: 20px;
        padding: 10px 20px;
        background-color: #007bff;
        color: white;
        text-decoration: none;
        border-radius: 4px;
    }

    .back-link:hover {
        background-color: #0056b3;
    }
</style>
</head>
<body>
    <div class="container">
        <h2><img alt="Calculator icon" data-bbox="161 388 181 403"/> Calculation Result</h2>

        @{
            double num1 = Convert.ToDouble(ViewData["Number1"] ?? 0);
            double num2 = Convert.ToDouble(ViewData["Number2"] ?? 0);
            double result = Convert.ToDouble(ViewData["Result"] ?? 0);
        }

        <div class="result-box">
            <div class="formula">Your Calculation:</div>
            <div class="calculation">
                @num1 + @num2 = @result
            </div>
            <hr />
            <div style="font-size: 16px; color: #666;">
                @num1 plus @num2 equals <strong style="color:#2e7d32; font-
size:20px;">@result</strong>
            </div>
        </div>

        <a href="/Calculator" class="back-link"><img alt="Refresh icon" data-bbox="208 651 228 666"/> Perform New Calculation</a>
    </div>
</body>
</html>

```

Views/ Index.cshtml

```

@{
    Layout = null;
}

<!DOCTYPE html>
<html>
<head>
    <title>Simple Calculator</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 50px;
            background-color: #f0f0f0;
        }

        .container {

```



```

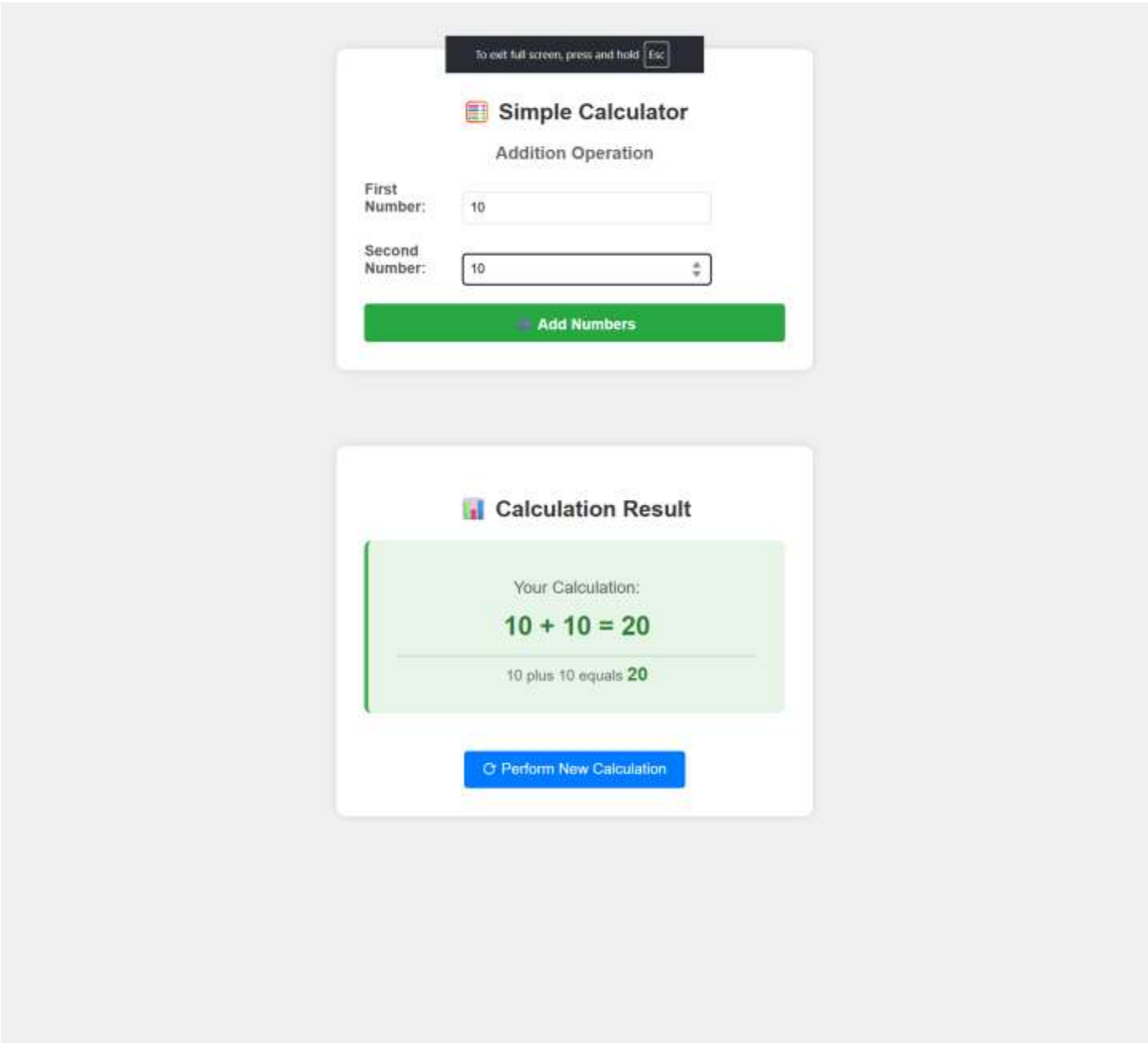
        width: 450px;
        padding: 30px;
        background-color: white;
        border-radius: 10px;
        box-shadow: 0 0 10px rgba(0,0,0,0.1);
        margin: auto;
    }
    h2 {
        color: #333;
        text-align: center;
    }
    .form-group {
        margin-bottom: 20px;
    }
    label {
        display: inline-block;
        width: 100px;
        font-weight: bold;
        color: #555;
    }
    input {
        padding: 8px;
        width: 250px;
        border: 1px solid #ccc;
        border-radius: 4px;
        font-size: 14px;
    }
    button {
        width: 100%;
        padding: 10px;
        background-color: #28a745;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
        font-size: 16px;
        font-weight: bold;
    }
    button:hover {
        background-color: #218838;
    }

    .result-box {
        margin-top: 20px;
        text-align: center;
    }
}
</style>
</head>
<body>
    <div class="container">
        <h2>☰ Simple Calculator</h2>
        <h3 style="text-align:center; color:#666;">Addition Operation</h3>

        <form method="post" action="/Calculator/Add">
            <div class="form-group">
                <label>First Number:</label>
                <input type="number" name="num1" step="any" required placeholder="Enter first
number" />
            </div>
            <div class="form-group">
                <label>Second Number:</label>
                <input type="number" name="num2" step="any" required placeholder="Enter second
number" />
            </div>
            <button type="submit">⊕ Add Numbers</button>
        </form>
    </div>
</body>
</html>

```

OUTPUT:



Problem 3 (Level-2): Product Entry with List Display

Scenario:

An admin wants to add multiple products and view them in a list on the same page.

Requirements:

1. Create a form to input:
 - Product Name
 - Price
 - Quantity
2. On submission:
 - Add product to a List
 - Display all products in tabular format
3. Use ViewBag to store and display list

Technical Constraints

1. Use Attribute-based routing
2. Use HttpPost for adding data
3. Maintain list temporarily (no database)
4. Use static list or TempData alternative NOT allowed

Expectations

1. Data persists across multiple submissions (within session scope)
2. Table updates dynamically after each submission
3. Clean UI separation (form + table)

Learning Outcome

1. Managing **collections using ViewBag/ViewData**
2. Handling repeated form submissions
3. Understanding limitations of ViewBag

CODE:

```
📁 Controllers/ ProductController.cs

using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;

namespace ProductManagement.Controllers{
```

```

public class ProductController : Controller
{
    private static List<Product> products = new List<Product>();

    [Route("")]
    [Route("Product")]
    [Route("Product/Index")]
    [HttpGet]
    public IActionResult Index()
    {
        // Store products list in ViewBag to display in table
        ViewBag.ProductList = products;
        return View();
    }

    [Route("Product/Add")]
    [HttpPost]
    public IActionResult Add(string productName, decimal price, int quantity)
    {
        // Add new product to the list
        Product newProduct = new Product
        {
            Id = products.Count + 1,
            ProductName = productName,
            Price = price,
            Quantity = quantity
        };

        products.Add(newProduct);

        // Store updated list in ViewBag
        ViewBag.ProductList = products;
        ViewBag.Message = $"Product '{productName}' added successfully!";

        return View("Index");
    }
}

public class Product
{
    public int Id { get; set; }
    public string ProductName { get; set; }
    public decimal Price { get; set; }
    public int Quantity { get; set; }
    public decimal TotalValue => Price * Quantity;
}

```

Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Product}/{action=Index}/{id?}");

```

```
app.Run();
```

 Views/ Index.cshtml

```
@{
    Layout = null;
    var products = ViewBag.ProductList as List<ProductManagement.Controllers.Product> ?? new
    List<ProductManagement.Controllers.Product>();
}

<!DOCTYPE html>
<html>
<head>
    <title>Product Management</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            margin: 30px;
            background-color: #f5f5f5;
        }
        .container {
            max-width: 1200px;
            margin: auto;
            background-color: white;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 0 20px rgba(0,0,0,0.1);
        }
        h2 {
            color: #333;
            border-bottom: 3px solid #007bff;
            padding-bottom: 10px;
        }
        .form-container {
            background-color: #f8f9fa;
            padding: 20px;
            border-radius: 8px;
            margin-bottom: 30px;
        }
        .form-group {
            margin-bottom: 15px;
        }
        label {
            display: inline-block;
            width: 120px;
            font-weight: bold;
            color: #555;
        }
        input {
            padding: 8px;
            width: 250px;
            border: 1px solid #ccc;
            border-radius: 4px;
        }
        button {
            background-color: #28a745;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            font-size: 16px;
            margin-left: 120px;
        }
        button:hover {
            background-color: #218838;
        }
        .message {
            background-color: #d4edda;
            color: #155724;
        }
    </style>
</head>
<body>
    <div class="container">
        <h2>Product Management</h2>
        <div class="form-container">
            <div class="form-group">
                <label>Name</label>
                <input type="text"/>
            </div>
            <div class="form-group">
                <label>Email</label>
                <input type="text"/>
            </div>
            <div class="form-group">
                <label>Phone</label>
                <input type="text"/>
            </div>
            <div class="form-group">
                <label>Address</label>
                <input type="text"/>
            </div>
            <button type="button">Add Product</button>
        </div>
        <div class="message">
            <div class="container">
                <div class="form-group">
                    <label>Name</label>
                    <input type="text"/>
                </div>
                <div class="form-group">
                    <label>Email</label>
                    <input type="text"/>
                </div>
                <div class="form-group">
                    <label>Phone</label>
                    <input type="text"/>
                </div>
                <div class="form-group">
                    <label>Address</label>
                    <input type="text"/>
                </div>
                <button type="button">Add Product</button>
            </div>
        </div>
    </div>
</body>
</html>
```

```

padding: 10px;
border-radius: 4px;
margin-bottom: 20px;
border-left: 4px solid #28a745;
}
table {
width: 100%;
border-collapse: collapse;
margin-top: 20px;
}
th, td {
border: 1px solid #ddd;
padding: 12px;
text-align: left;
}
th {
background-color: #007bff;
color: white;
font-weight: bold;
}
tr:nth-child(even) {
background-color: #f2f2f2;
}
tr:hover {
background-color: #e9ecef;
}
.no-data {
text-align: center;
padding: 40px;
color: #999;
font-style: italic;
}
.total-row {
background-color: #ffeaa7;
font-weight: bold;
}
}
</style>
</head>
<body>
<div class="container">
<h2>📦 Product Management System</h2>

<div class="form-container">
<h3>Add New Product</h3>

@if (ViewBag.Message != null)
{
<div class="message">
☒ @ViewBag.Message
</div>
}

<form method="post" action="/Product/Add">
<div class="form-group">
<label>Product Name:</label>
<input type="text" name="productName" required placeholder="Enter product
name" />
</div>
<div class="form-group">
<label>Price ($):</label>
<input type="number" name="price" step="0.01" required placeholder="0.00" />
</div>
<div class="form-group">
<label>Quantity:</label>
<input type="number" name="quantity" required placeholder="0" />
</div>
<button type="submit">+ Add Product</button>
</form>
</div>

```

```

<h3>Product List</h3>
@if (products.Any()) {
    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Product Name</th>
                <th>Price ($)</th>
                <th>Quantity</th>
                <th>Total Value ($)</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var product in products) {
                <tr>
                    <td>@product.Id</td>
                    <td>@product.ProductName</td>
                    <td>@product.Price.ToString("F2")</td>
                    <td>@product.Quantity</td>
                    <td>@product.TotalValue.ToString("F2")</td>
                </tr>
            }
        </tbody>
        <tfoot>
            <tr class="total-row">
                <td colspan="4" style="text-align: right;"><strong>Total
Products:</strong></td>
                <td><strong>@products.Count</strong></td>
            </tr>
        </tfoot>
    </table>
}
else {
    <div class="no-data">
        📦 No products added yet. Add your first product using the form above!
    </div>
}
</div>
</body>
</html>

```

OUTPUT:

 **Product Management System**

Add New Product

 Product 'Table' added successfully!

Product Name:

Price (\$):

Quantity:

Add Product

Product List

ID	Product Name	Price (\$)	Quantity	Total Value (\$)
1	Table	60.00	1	60.00
Total Products:				1

Problem 4 (Level-2): Feedback Form with Conditional Message

Scenario:

A website collects user feedback and displays different messages based on rating.

Requirements:

1. Create a feedback form:
 - Name
 - Comments
 - Rating (1-5)
2. Submit using HttpPost
3. After submission:
 - Show "Thank You" message if rating ≥ 4
 - Show "We will improve" message if rating < 4
4. Use ViewData to pass message

Technical Constraints

1. Use Attribute routing
2. Do NOT use TempData/Session
3. Do NOT use database
4. Must use ViewData for message handling

Expectations

1. Correct conditional logic
2. Proper use of ViewData
3. Clean routing structure

Learning Outcome

1. Conditional rendering using server-side logic
2. Using ViewData for dynamic UI behavior
3. Reinforcing form handling concepts

CODE:

 Controllers/ `FeedbackController.cs`

`using Microsoft.AspNetCore.Mvc;`


```

namespace FeedbackSystem.Controllers
{
    public class FeedbackController : Controller
    {
        [Route("")]
        [Route("Feedback")]
        [Route("Feedback/Index")]
        [HttpGet]
        public IActionResult Index()
        {
            return View();
        }

        [Route("Feedback/Submit")]
        [HttpPost]
        public IActionResult Submit(string name, string comments, int rating)
        {
            // Store feedback data in ViewData
            ViewData["Name"] = name;
            ViewData["Comments"] = comments;
            ViewData["Rating"] = rating;

            // Conditional message based on rating
            if (rating >= 4)
            {
                ViewData["Message"] = "Thank You! 🌟 We really appreciate your positive feedback!";
                ViewData["MessageType"] = "success";
            }
            else
            {
                ViewData["Message"] = "We will improve! 😞 Thank you for your honest feedback.";
                ViewData["MessageType"] = "warning";
            }

            return View("FeedbackResult");
        }
    }
}

```

Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Feedback}/{action=Index}/{id?}");

app.Run();

```

Views/FeedBack/Index.cshtml

```

@{
    Layout = null;
}

```

```
<!DOCTYPE html>
<html>
<head>
  <title>Feedback Form</title>
  <style>
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      margin: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
    }

    .container {
      max-width: 600px;
      margin: auto;
      background-color: white;
      padding: 40px;
      border-radius: 15px;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
    }

    h2 {
      color: #333;
      text-align: center;
      margin-bottom: 30px;
    }

    .form-group {
      margin-bottom: 20px;
    }

    label {
      display: block;
      font-weight: bold;
      margin-bottom: 8px;
      color: #555;
    }

    input[type="text"], textarea {
      width: 100%;
      padding: 10px;
      border: 2px solid #ddd;
      border-radius: 8px;
      font-size: 14px;
      font-family: inherit;
      box-sizing: border-box;
    }

    textarea {
      resize: vertical;
      min-height: 100px;
    }

    .rating {
      display: flex;
      gap: 15px;
      margin-top: 10px;
      flex-wrap: wrap;
    }

    .rating-option {
      flex: 1;
      text-align: center;
      min-width: 60px;
    }

    .rating-option input {
      display: none;
    }
  </style>
</head>
</html>
```

```

        .rating-option label {
            display: inline-block;
            padding: 10px 20px;
            background-color: #f0f0f0;
            border-radius: 8px;
            cursor: pointer;
            transition: all 0.3s;
            font-weight: normal;
            width: 100%;
            box-sizing: border-box;
        }

        .rating-option input:checked + label {
            background-color: #ffc107;
            color: #333;
            font-weight: bold;
            transform: scale(1.05);
        }

        .rating-option label:hover {
            background-color: #ffc107;
            transform: scale(1.05);
        }

        button {
            width: 100%;
            padding: 12px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            border: none;
            border-radius: 8px;
            font-size: 16px;
            font-weight: bold;
            cursor: pointer;
            transition: transform 0.2s;
        }

        button:hover {
            transform: translateY(-2px);
        }

        .info {
            text-align: center;
            margin-top: 20px;
            color: #666;
            font-size: 14px;
        }
    }
</style>
</head>
<body>
    <div class="container">
        <h2>📝 We Value Your Feedback!</h2>

        <form method="post" action="/Feedback/Submit">
            <div class="form-group">
                <label>Your Name:</label>
                <input type="text" name="name" required placeholder="Enter your name" />
            </div>

            <div class="form-group">
                <label>Your Comments:</label>
                <textarea name="comments" required placeholder="Share your thoughts with us..."></textarea>
            </div>

            <div class="form-group">
                <label>Rating (1-5):</label>
                <div class="rating">
                    <div class="rating-option">
                        <input type="radio" id="rating1" name="rating" value="1" required />

```

```

        <label for="rating1">★ 1</label>
    </div>
    <div class="rating-option">
        <input type="radio" id="rating2" name="rating" value="2" />
        <label for="rating2">★ ★ 2</label>
    </div>
    <div class="rating-option">
        <input type="radio" id="rating3" name="rating" value="3" />
        <label for="rating3">★ ★ ★ 3</label>
    </div>
    <div class="rating-option">
        <input type="radio" id="rating4" name="rating" value="4" />
        <label for="rating4">★ ★ ★ ★ 4</label>
    </div>
    <div class="rating-option">
        <input type="radio" id="rating5" name="rating" value="5" />
        <label for="rating5">★ ★ ★ ★ ★ 5</label>
    </div>
</div>
</div>

    <button type="submit">Submit Feedback</button>
</form>

<div class="info">
    💡 Your feedback helps us serve you better!
</div>
</div>
</body>
</html>

```

Views/FeedBack/FeedBackResult.cshtml

```

@{
    Layout = null;
    string message = ViewData["Message"]?.ToString() ?? "";
    string messageType = ViewData["MessageType"]?.ToString() ?? "";
    string name = ViewData["Name"]?.ToString() ?? "";
    string comments = ViewData["Comments"]?.ToString() ?? "";
    int rating = ViewData["Rating"] != null ? Convert.ToInt32(ViewData["Rating"]) : 0;
}

<!DOCTYPE html>
<html>
<head>
    <title>Feedback Received</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            margin: 50px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
        }

        .container {
            max-width: 600px;
            margin: auto;
            background-color: white;
            padding: 40px;
            border-radius: 15px;
            box-shadow: 0 20px 60px rgba(0,0,0,0.3);
            text-align: center;
        }

        .success-message {
            background-color: #d4edda;
            color: #155724;
            padding: 20px;
            border-radius: 10px;
            margin: 20px 0;
        }
    </style>

```

```

        border-left: 5px solid #28a745;
        animation: slideIn 0.5s ease;
    }

    .warning-message {
        background-color: #fff3cd;
        color: #856404;
        padding: 20px;
        border-radius: 10px;
        margin: 20px 0;
        border-left: 5px solid #ffc107;
        animation: slideIn 0.5s ease;
    }
    /* Use double @@ to escape the @@ symbol in Razor */
    @@keyframes slideIn {
        from {
            transform: translateY(-20px);
            opacity: 0;
        }

        to {
            transform: translateY(0);
            opacity: 1;
        }
    }

    .feedback-details {
        background-color: #f8f9fa;
        padding: 20px;
        border-radius: 10px;
        margin: 20px 0;
        text-align: left;
    }

    .rating-display {
        font-size: 24px;
        margin: 10px 0;
    }

    button {
        padding: 12px 30px;
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        color: white;
        border: none;
        border-radius: 8px;
        font-size: 16px;
        font-weight: bold;
        cursor: pointer;
        margin-top: 20px;
        transition: transform 0.2s;

        button:hover {
            transform: translateY(-2px);
        }
    }

    .emoji {
        font-size: 50px;
        margin: 10px 0;
    }
}
</style>
</head>
<body>
    <div class="container">
        <div class="emoji">
            @if (rating >= 4)
            {
                <span>🌟</span>
            }
            else

```

```

        {
            <span>ⓧ</span>
        }
    </div>

    <h2>Feedback Received!</h2>

    <div class="@messageType == 'success' ? 'success-message' : 'warning-message'">
        <h3>@message</h3>
    </div>

    <div class="feedback-details">
        <h3>Your Feedback Summary:</h3>
        <p><strong>Name:</strong> @name</p>
        <p><strong>Comments:</strong> @comments</p>
        <p>
            <strong>Rating:</strong>
            <span class="rating-display">
                @for (int i = 1; i <= 5; i++)
                {
                    if (i <= rating)
                    {
                        <span>★</span>
                    }
                    else
                    {
                        <span>☆</span>
                    }
                }
                (@rating/5)
            </span>
        </p>
    </div>

    <button onclick="location.href='/Feedback'">Submit Another Feedback</button>
</div>
</body>
</html>

```

OUTPUT:

We Value Your Feedback!

Your Name:

Your Comments:

Rating (1-5):

☐ 1
 ☐ 2
 ☐ 3
 ☒ 4
 ☐ 5

[Submit Feedback](#)

💡 Your feedback helps us serve you better!



Feedback Received!

Thank You! 🌟 We really appreciate your positive feedback!

Your Feedback Summary:

Name: Keerthivasan

Comments: It was Good

Rating: ★ ★ ★ ★ ☆ (4/5)

[Submit Another Feedback](#)